

TAREA PRÁCTICA: PERSISTENCIA CON ARCHIVOS DE TEXTO

Nombre: Guanotoa Escobar Karla Leonela

Fecha: 02/05/2026

Link de GitHub: <https://github.com/Karla-ge08/TAREA2.2>

Objetivo General

- Aplicar el manejo de archivos para leer, procesar y analizar datos en C++ y Java.

Objetivos Específicos

- Implementar la lectura de datos externos desde el archivo `estudiantes.txt` mediante el uso de `ifstream` en C++ y la clase `Scanner` en Java.
- Desarrollar lógica de programación para el cálculo de promedios individuales y generales, así como la contabilización de estudiantes aprobados bajo condiciones específicas.
- Aplicar estructuras de control (`while`, `if-else`) y manejo de excepciones (`try-catch`) para procesar información de productos y estudiantes de forma iterativa y segura.

Ejercicio 1: Estudiantes.

Archivo `estudiantes.txt`:

Ana 8 9 7

Carlos 5 6 4

Lucia 10 9 10

Pedro 6 7 5

Actividad para el estudiante:

1. Ejecutar el programa
2. Modificar el archivo `estudiantes.txt`
3. Agregar un estudiante nuevo
4. Cambiar la condición:
 - $\geq 9 \rightarrow$ Excelente
 - $7-8 \rightarrow$ Bueno
 - $< 7 \rightarrow$ Reprobado

1) Análisis del problema.

El problema consiste en desarrollar una aplicación capaz de leer un archivo de texto externo denominado `estudiantes.txt`, el cual contiene un listado de nombres y sus

respectivas calificaciones. El sistema debe procesar cada registro para calcular el promedio individual de tres notas y determinar su estado académico mediante una escala cualitativa (Excelente, Bueno, Reprobado). Finalmente, se debe calcular y mostrar el promedio general de todo el curso y el total de registros procesados.

2) Variables de entrada, proceso y salida.

Entrada

nombre

n1, n2, n3

Proceso

promedio

sumaPromedios

total

Salida

Estado (Excelente, Bueno, Reprobado).

promedioGeneral

sumaPromedios

3) Algoritmo.

1. Definir la ruta del archivo físico `estudiantes.txt`.
2. Verificar la existencia del archivo; si no existe, mostrar un error y terminar.
3. Inicializar el contador `total` y el acumulador `sumaPromedios` en cero.
4. Mientras existan datos en el archivo, repetir:
 - Leer `nombre`, `n1`, `n2` y `n3`.
 - Calcular $\text{promedio} = (n1 + n2 + n3) / 3$.
 - Si `promedio` ≥ 9 , asignar estado "Excelente".
 - Si `promedio` < 9 y `promedio` ≥ 7 , asignar estado "Bueno".
 - Si `promedio` < 7 , asignar estado "Reprobado".
 - Mostrar los resultados del estudiante en pantalla.
 - Sumar el `promedio` al acumulador y aumentar el contador.
5. Si el contador es mayor a cero, calcular y mostrar el promedio general.
6. Cerrar el archivo.

4) Pseudocódigo.

Algoritmo ProcesarEstudiantes

Definir nombre Como Cadena

Definir n1, n2, n3, promedio, sumaPromedios Como Real

Definir total Como Entero

total <- 0

sumaPromedios <- 0

Escribir "Ingrese nombre (o use la palabra fin para salir):"

Leer nombre

Mientras nombre <> "fin" Hacer

 Escribir "Ingrese nota 1:"

 Leer n1

 Escribir "Ingrese nota 2:"

 Leer n2

 Escribir "Ingrese nota 3:"

 Leer n3

 promedio <- (n1 + n2 + n3) / 3

 Si promedio >= 9 Entonces

 Escribir "Nombre: ", nombre, " Promedio: ", promedio, " - Excelente"

 Sino

 Si promedio >= 7 Entonces

 Escribir "Nombre: ", nombre, " Promedio: ", promedio, " - Bueno"

 Sino

 Escribir "Nombre: ", nombre, " Promedio: ", promedio, " -

Reprobado"

 FinSi

 FinSi

sumaPromedios <- sumaPromedios + promedio

total <- total + 1

Escribir "Ingrese siguiente nombre (o fin):"

Leer nombre

FinMientras

Si total > 0 Entonces

 Escribir "-----"

 Escribir "Total de estudiantes: ", total

 Escribir "Promedio general: ", sumaPromedios / total

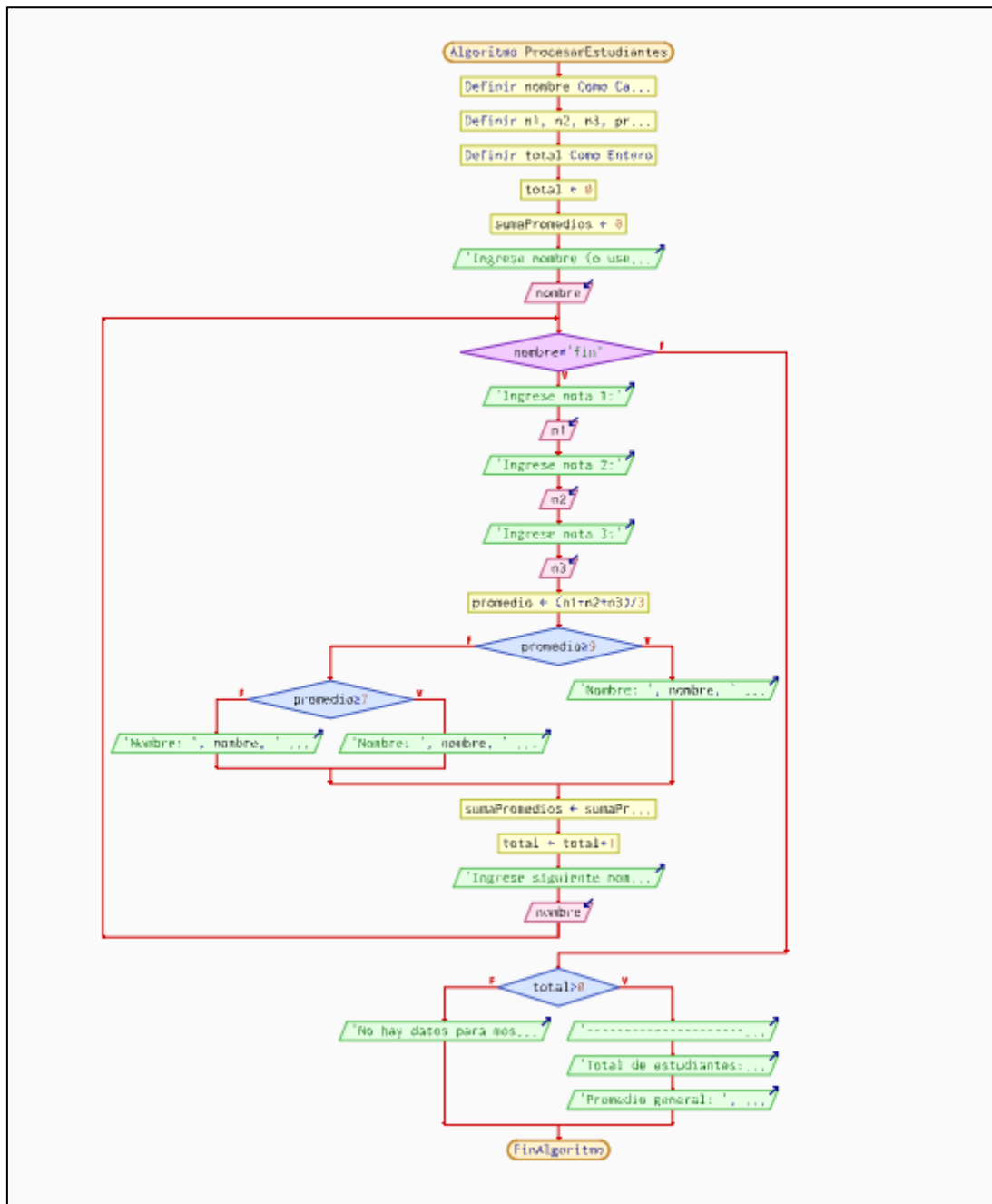
Sino

 Escribir "No hay datos para mostrar"

FinSi

FinAlgoritmo

5) Diagrama de flujo.



6) Código en C++.

```
#include <iostream>
#include <fstream>
#include <string>
```

```
using namespace std;
```

```
int main() {
    ifstream archivo("estudiantes.txt");
```

```

string nombre;
double n1, n2, n3, promedio;
int total = 0, aprobados = 0;
double suma = 0;

if (!archivo.is_open()) return 1;

while (archivo >> nombre >> n1 >> n2 >> n3) {
    promedio = (n1 + n2 + n3) / 3.0;

    cout << "Nombre: " << nombre << " | Promedio: " << promedio << " |
Estado: ";

    if (promedio >= 9) {
        cout << "Excelente" << endl;
        aprobados++;
    } else if (promedio >= 7) {
        cout << "Bueno" << endl;
        aprobados++;
    } else {
        cout << "Reprobado" << endl;
    }

    suma += promedio;
    total++;
}

if (total > 0) {
    cout << "-----" << endl;
    cout << "Promedio general: " << suma / total << endl;
}

archivo.close();
return 0;
}

```

7) Código en JAVA.

/*

* Click <nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt> to change this license

* Click <nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java> to edit this template

```

*/

/**
 *
 * @author Karla
 */

import java.io.File;

import java.io.FileNotFoundException;

import java.util.Scanner;


public class T21_MA {

    public static void main(String[] args) {

        // SOLUCIÓN: Ruta exacta hacia tu Escritorio

        // Asegúrate de cambiar USUARIO por el nombre que viste en tu PC

        String ruta = "C:/Users/USUARIO/Desktop/TRABAJOS UTA KARLA
GUANOTOA/estudiantes.txt";

        File archivo = new File(ruta);

        if (!archivo.exists()) {

            System.out.println("Error: No se encontro el archivo.");

            System.out.println("Buscando en: " + ruta);

            return;

        }

        int totalEstudiantes = 0;

        double sumaPromedios = 0;

```

```

try {

    Scanner lector = new Scanner(archivo);

    System.out.println("--- REPORTE DE CALIFICACIONES ---");

    while (lector.hasNext()) {

        String nombre = lector.next();

        double n1 = lector.nextDouble();

        double n2 = lector.nextDouble();

        double n3 = lector.nextDouble();

        double promedio = (n1 + n2 + n3) / 3.0;

        System.out.print("Nombre: " + nombre + " | Promedio: " +
String.format("%.2f", promedio));

        // Clasificación según la actividad para estudiantes

        if (promedio >= 9) {

            System.out.println(" | Estado: Excelente");

        } else if (promedio >= 7) {

            System.out.println(" | Estado: Bueno");

        } else {

            System.out.println(" | Estado: Reprobado");

        }

        sumaPromedios += promedio;

        totalEstudiantes++;
    }
}

```

```

    }

    if (totalEstudiantes > 0) {

        System.out.println("-----");

        System.out.println("Total de estudiantes: " + totalEstudiantes);

        System.out.println("Promedio general del curso: " +
String.format("%.2f", (sumaPromedios / totalEstudiantes)));

    }

    lector.close();

} catch (FileNotFoundException e) {

    System.out.println("Error: No se pudo abrir el archivo.");

} catch (Exception e) {

    System.out.println("Error: Verifique el formato de las notas (puntos o
comas).");

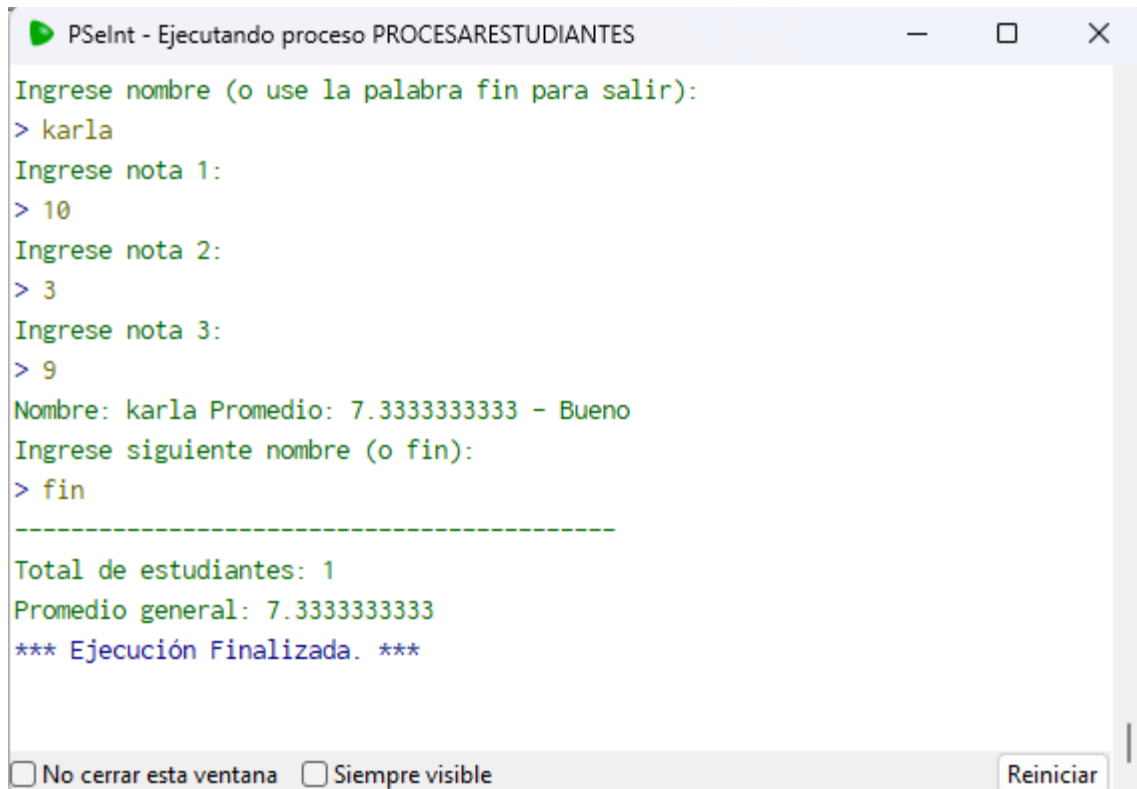
}

}

}

```

8) Prueba de solución.



```
PSeInt - Ejecutando proceso PROCESARESTUDIANTES

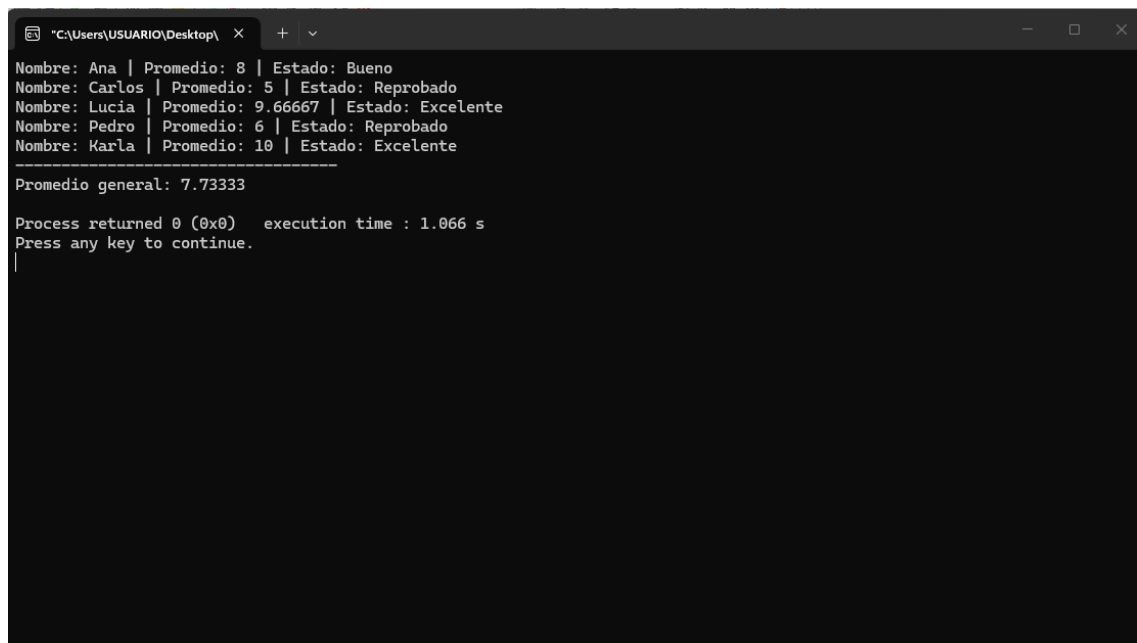
Ingrese nombre (o use la palabra fin para salir):
> karla
Ingrese nota 1:
> 10
Ingrese nota 2:
> 3
Ingrese nota 3:
> 9
Nombre: karla Promedio: 7.333333333 - Bueno
Ingrese siguiente nombre (o fin):
> fin

-----

Total de estudiantes: 1
Promedio general: 7.333333333
*** Ejecución Finalizada. ***

☐ No cerrar esta ventana ☐ Siempre visible Reiniciar
```

El pseudocódigo se ejecuta de manera adecuada.

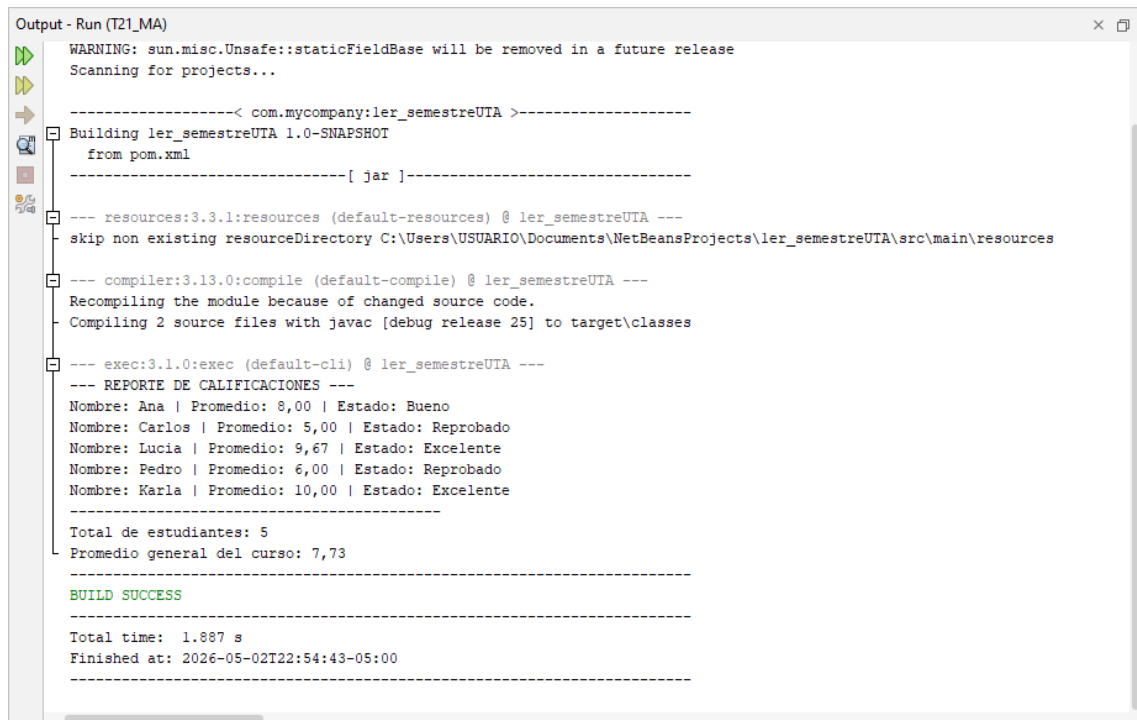


```
"C:\Users\USUARIO\Desktop\ x + v

Nombre: Ana | Promedio: 8 | Estado: Bueno
Nombre: Carlos | Promedio: 5 | Estado: Reprobado
Nombre: Lucia | Promedio: 9.66667 | Estado: Excelente
Nombre: Pedro | Promedio: 6 | Estado: Reprobado
Nombre: Karla | Promedio: 10 | Estado: Excelente
-----
Promedio general: 7.73333

Process returned 0 (0x0) execution time : 1.066 s
Press any key to continue.
```

El código compila de manera correcta en C++ encontrando el archivo estudiantes sin inconvenientes.



```
Output - Run (T21_MA)
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release
Scanning for projects...

-----< com.mycompany:ler_semestreUTA >-----
Building ler_semestreUTA 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----

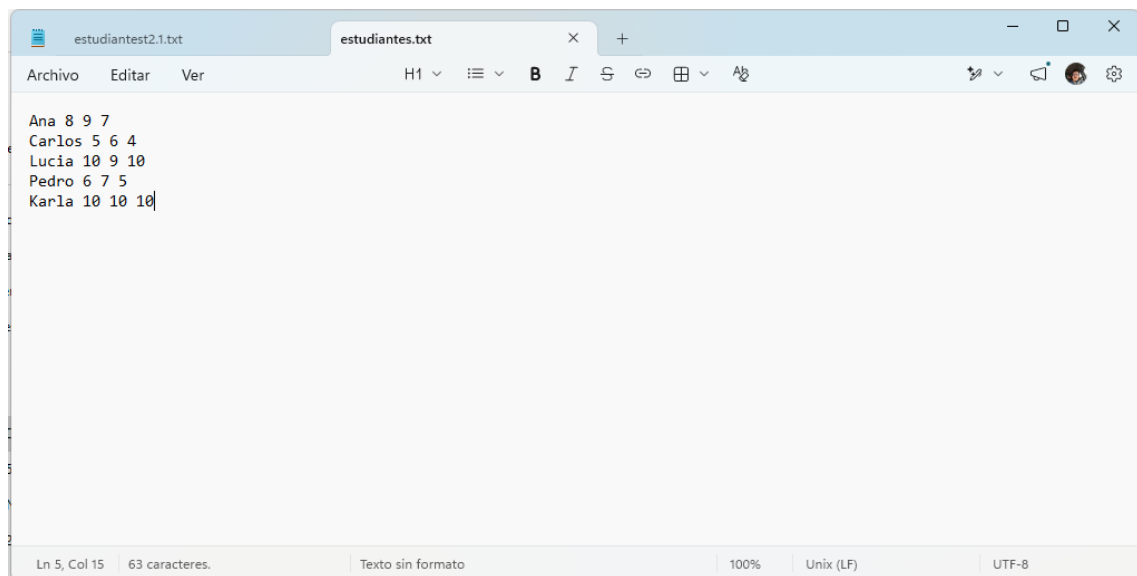
--- resources:3.3.1:resources (default-resources) @ ler_semestreUTA ---
skip non existing resourceDirectory C:\Users\USUARIO\Documents\NetBeansProjects\ler_semestreUTA\src\main\resources

--- compiler:3.13.0:compile (default-compile) @ ler_semestreUTA ---
Recompiling the module because of changed source code.
Compiling 2 source files with javac [debug release 25] to target\classes

--- exec:3.1.0:exec (default-cli) @ ler_semestreUTA ---
--- REPORTE DE CALIFICACIONES ---
Nombre: Ana | Promedio: 8,00 | Estado: Bueno
Nombre: Carlos | Promedio: 5,00 | Estado: Reprobado
Nombre: Lucia | Promedio: 9,67 | Estado: Excelente
Nombre: Pedro | Promedio: 6,00 | Estado: Reprobado
Nombre: Karla | Promedio: 10,00 | Estado: Excelente
-----
Total de estudiantes: 5
Promedio general del curso: 7,73
-----
BUILD SUCCESS
-----
Total time: 1.887 s
Finished at: 2026-05-02T22:54:43-05:00
-----
```

El código compila de manera adecuada en JAVA pero se necesitó abrir una ruta específica para que el código encuentre el archivo txt. Sin inconvenientes.

9) Archivo de texto.



```
estudiante21.txt  estudiantes.txt
Archivo  Editar  Ver  H1  B  I  S  A  100%  Unix (LF)  UTF-8

Ana 8 9 7
Carlos 5 6 4
Lucia 10 9 10
Pedro 6 7 5
Karla 10 10 10

Ln 5, Col 15  63 caracteres.  Texto sin formato  100%  Unix (LF)  UTF-8
```

Visualización del archivo txt.

Ejercicio 2: Productos.

Archivo productos.txt:

Laptop 850 2

Mouse 15 5

Teclado 30 3

Monitor 180 2

Ventilador 50 9

Implementar en C++ y Java cálculo de subtotal, total y producto mayor.

1) Análisis del problema.

El programa debe leer datos de un archivo llamado `productos.txt`, donde cada registro contiene el nombre de un producto, su precio unitario y la cantidad comprada. El objetivo es calcular el subtotal por cada producto ($\text{precio} \times \text{cantidad}$), sumar todos estos valores para obtener un total general e identificar cuál es el producto que generó el mayor gasto (el subtotal más alto).

2) Variables de entrada, proceso y salida.

Entrada

`nombreProdCadena` (Laptop, Mouse, etc.).

`precioValor`

`cantidadValor`

Proceso

`subtotal` Multiplicación de `precio * cantidad`.

`totalVentaAcumulador` de todos los subtotales.

`maxSubtotalVariable` para comparar y guardar el subtotal más alto encontrado.

Salida

`totalFinalSuma` total de la compra.

`prodMayorNombre` del producto con el subtotal más elevado.

3) Algoritmo.

- 1) Abrir el archivo `productos.txt`.
- 2) Inicializar `totalVenta` en 0 y `maxSubtotal` en 0.
- 3) Mientras existan datos en el archivo:
 - Leer `nombreProd`, `precio` y `cantidad`.
 - Calcular `subtotal = precio * cantidad`.
 - Sumar `subtotal` a `totalVenta`.

- Si `subtotal > maxSubtotal`:
 - `maxSubtotal = subtotal.`
 - `prodMayor = nombreProd.`
 - Mostrar el nombre del producto y su subtotal.
- 4) Mostrar el `totalVenta` general.
 - 5) Mostrar el nombre del `prodMayor` y su valor.
 - 6) Cerrar archivo.

4) Pseudocódigo.

Algoritmo GestionProductos

Definir nombreProd, prodMayor Como Cadena

Definir precio, subtotal, totalVenta, maxSubtotal Como Real

Definir cantidad Como Entero

totalVenta <- 0

maxSubtotal <- 0

prodMayor <- ""

Escribir "Ingrese nombre del producto o escriba fin:"

Leer nombreProd

Mientras nombreProd <> "fin" Hacer

Escribir "Ingrese precio:"

Leer precio

Escribir "Ingrese cantidad:"

Leer cantidad

*subtotal <- precio * cantidad*

Escribir "Subtotal de ", nombreProd, ": ", subtotal

totalVenta <- totalVenta + subtotal

// Logica para encontrar el producto de mayor gasto

Si subtotal > maxSubtotal Entonces

maxSubtotal <- subtotal

prodMayor <- nombreProd

FinSi

Escribir "Ingrese el nombre del siguiente producto o fin:"

Leer nombreProd

FinMientras

Si totalVenta > 0 Entonces

Escribir "-----"

Escribir "Total de la venta es: ", totalVenta

Escribir "El producto con mayor gasto es: ", prodMayor

Escribir "Con un subtotal de: ", maxSubtotal

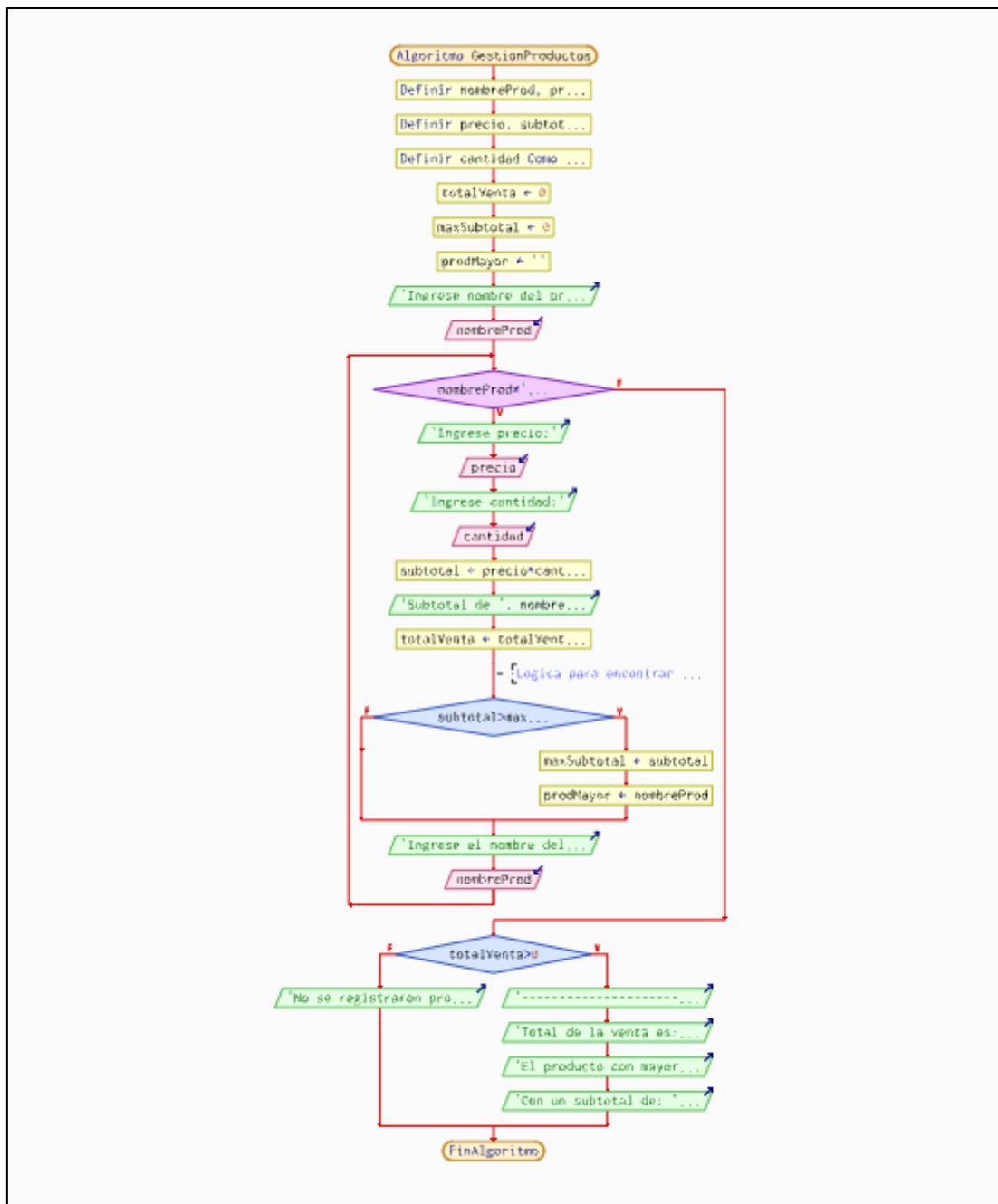
Sino

Escribir "No se registraron productos"

FinSi

FinAlgoritmo

5) Diagrama de flujo.



6) Código en C++.

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <iomanip>
```

```
using namespace std;
```

```

int main() {

    ifstream archivo("productos.txt");

    string nombreProd, prodMayor;

    double precio, subtotal, totalVenta = 0, maxSubtotal = -1;

    int cantidad;

    if (!archivo.is_open()) {

        cout << "--- ERROR: No se puede abrir 'productos.txt' ---" << endl;

        cout << "Asegurate de que el archivo este en la misma carpeta que este  
codigo." << endl;

        return 1;

    }

    cout << "--- DETALLE DE PRODUCTOS ---" << endl;

    cout << fixed << setprecision(2);

    while (archivo >> nombreProd >> precio >> cantidad) {

        subtotal = precio * cantidad;

        cout << "Producto: " << left << setw(12) << nombreProd

            << " | Subtotal: $" << subtotal << endl;

        totalVenta += subtotal;

        if (subtotal > maxSubtotal) {

            maxSubtotal = subtotal;

```

```

        prodMayor = nombreProd;
    }
}

cout << "-----" << endl;

if (totalVenta > 0) {
    cout << "TOTAL GENERAL DE LA VENTA: $" << totalVenta << endl;
    cout << "PRODUCTO CON MAYOR GASTO: " << prodMayor << " ($" <<
maxSubtotal << ")" << endl;
} else {
    cout << "AVISO: El archivo se abrio pero no se procesaron datos." << endl;
}

archivo.close();

return 0;
}

```

7) Código en JAVA.

```

/*
 * Click nbfs://nbhost/SystemFileSystem/Templates/Licenses/license-default.txt to
change this license
 * Click nbfs://nbhost/SystemFileSystem/Templates/Classes/Class.java to edit this
template
 */

/**
 *
 * @author USUARIO

```



```

        int cantidad = lector.nextInt();

        double subtotal = precio * cantidad;

        System.out.printf("Producto: %-12s | Subtotal: $%.2f%n",
nombreProd, subtotal);

        totalVenta += subtotal;

        if (subtotal > maxSubtotal) {
            maxSubtotal = subtotal;
            prodMayor = nombreProd;
        }
    }
}

lector.close();

System.out.println("-----");

if (totalVenta > 0) {

    System.out.printf("TOTAL GENERAL DE LA VENTA: $%.2f%n",
totalVenta);

    System.out.printf("PRODUCTO CON MAYOR GASTO: %s ($%.2f)%n",
prodMayor, maxSubtotal);

}

} catch (Exception e) {

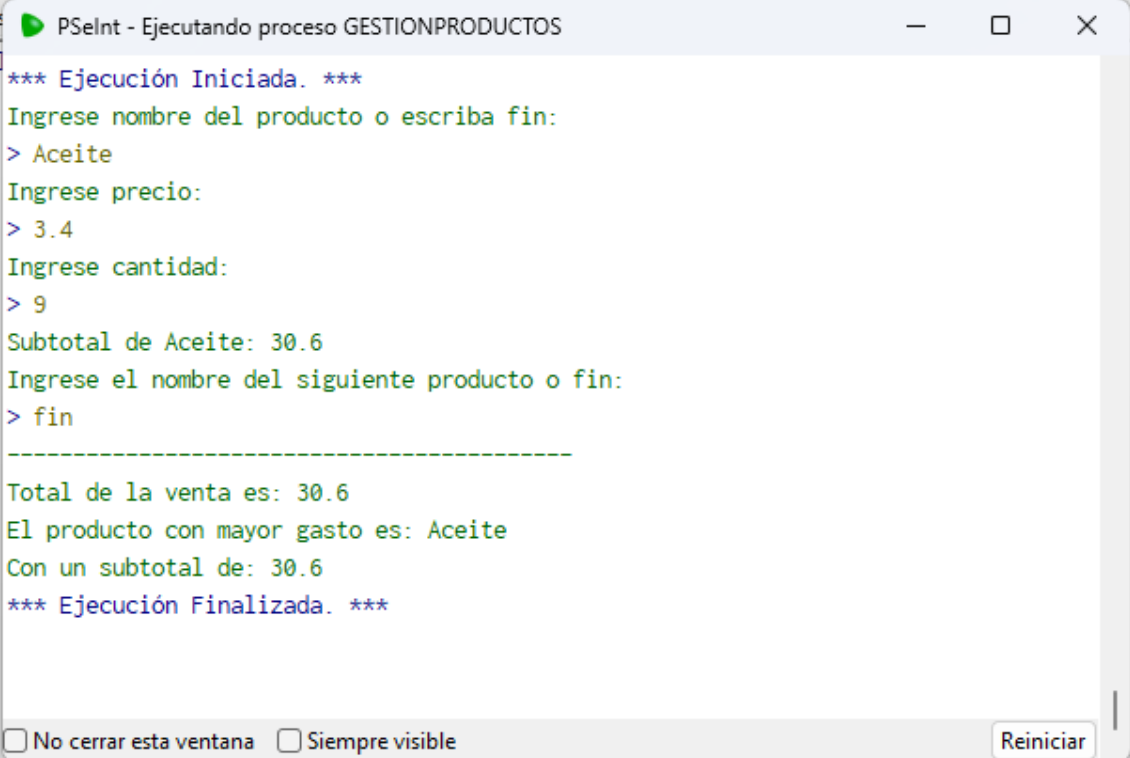
    System.out.println("Error al leer el archivo. Revisa la ruta o el formato.");

}

```

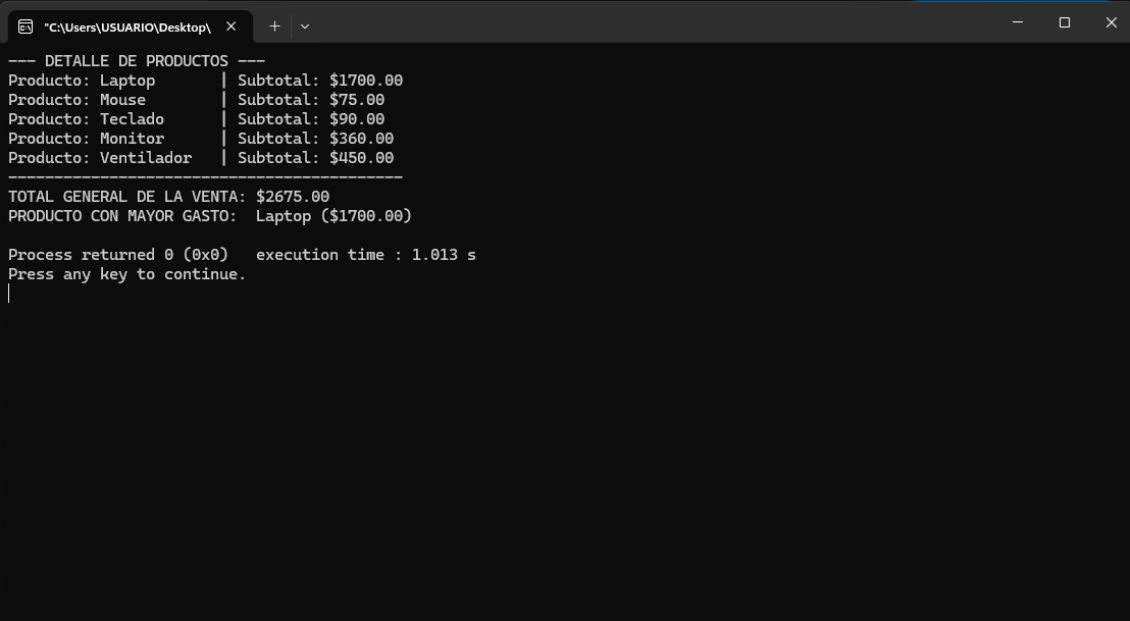
```
}  
  
}
```

8) Pruebas de solución.



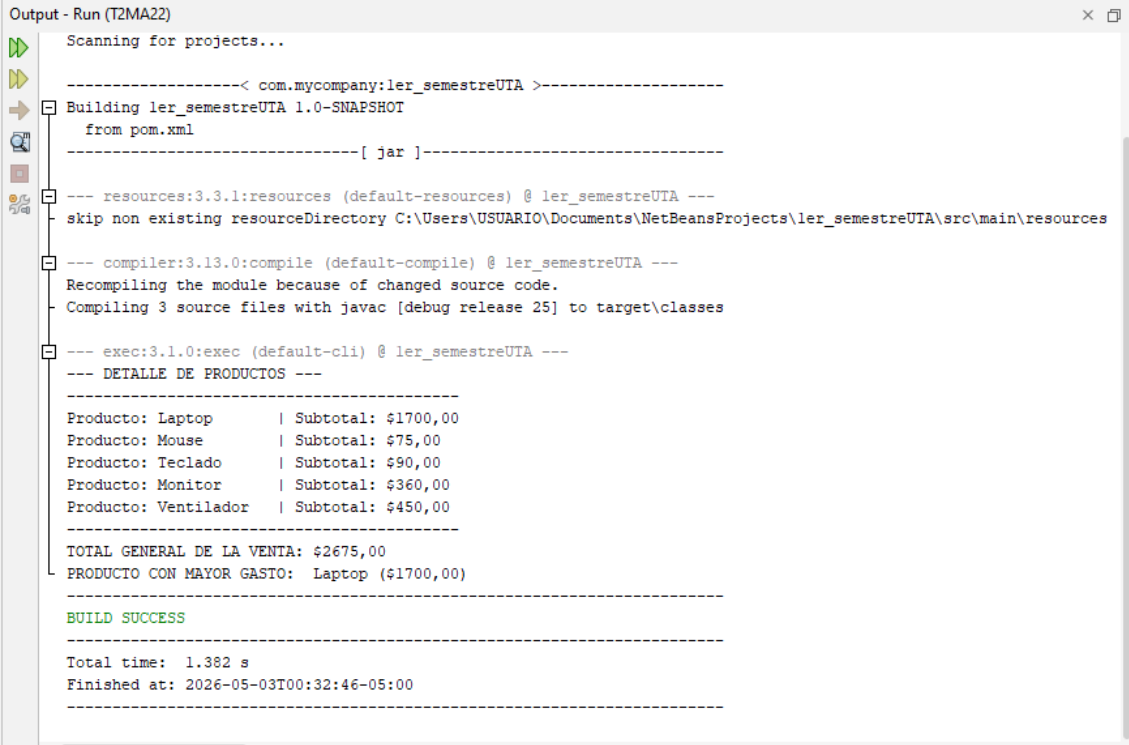
```
*** Ejecución Iniciada. ***  
Ingrese nombre del producto o escriba fin:  
> Aceite  
Ingrese precio:  
> 3.4  
Ingrese cantidad:  
> 9  
Subtotal de Aceite: 30.6  
Ingrese el nombre del siguiente producto o fin:  
> fin  
  
-----  
Total de la venta es: 30.6  
El producto con mayor gasto es: Aceite  
Con un subtotal de: 30.6  
*** Ejecución Finalizada. ***  
  
☐ No cerrar esta ventana ☐ Siempre visible Reiniciar
```

El pseudocódigo se ejecuta de manera adecuada sin errores.



```
--- DETALLE DE PRODUCTOS ---  
Producto: Laptop | Subtotal: $1700.00  
Producto: Mouse | Subtotal: $75.00  
Producto: Teclado | Subtotal: $90.00  
Producto: Monitor | Subtotal: $360.00  
Producto: Ventilador | Subtotal: $450.00  
-----  
TOTAL GENERAL DE LA VENTA: $2675.00  
PRODUCTO CON MAYOR GASTO: Laptop ($1700.00)  
  
Process returned 0 (0x0) execution time : 1.013 s  
Press any key to continue.
```

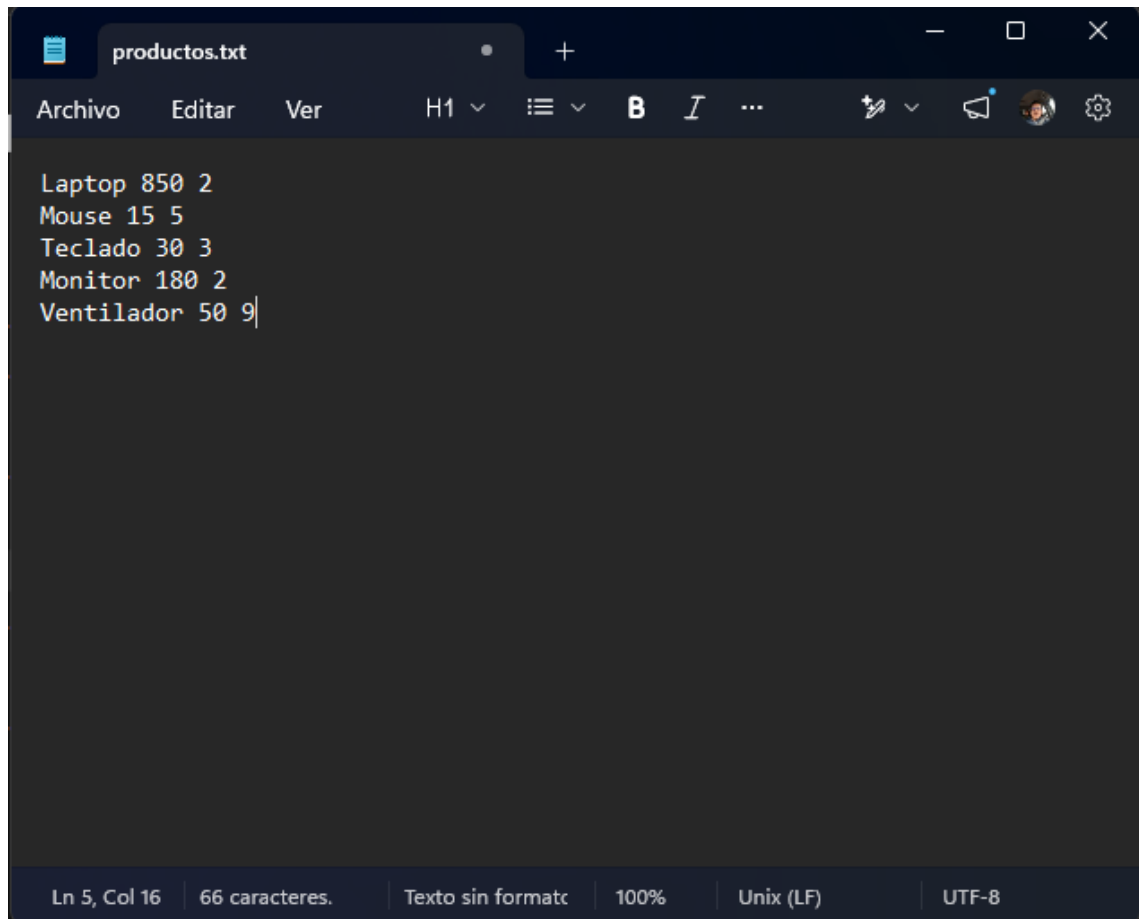
El código compila bien en C++ sin necesidad de usar una ruta, pero primero creamos el archivo txt (para que no exista inconvenientes)



```
Output - Run (T2MA22)
Scanning for projects...
-----< com.mycompany:ler_semestreUTA >-----
Building ler_semestreUTA 1.0-SNAPSHOT
from pom.xml
-----[ jar ]-----
--- resources:3.3.1:resources (default-resources) @ ler_semestreUTA ---
- skip non existing resourceDirectory C:\Users\USUARIO\Documents\NetBeansProjects\ler_semestreUTA\src\main\resources
--- compiler:3.13.0:compile (default-compile) @ ler_semestreUTA ---
Recompiling the module because of changed source code.
- Compiling 3 source files with javac [debug release 25] to target\classes
--- exec:3.1.0:exec (default-cli) @ ler_semestreUTA ---
--- DETALLE DE PRODUCTOS ---
-----
Producto: Laptop      | Subtotal: $1700,00
Producto: Mouse      | Subtotal: $75,00
Producto: Teclado    | Subtotal: $90,00
Producto: Monitor    | Subtotal: $360,00
Producto: Ventilador | Subtotal: $450,00
-----
TOTAL GENERAL DE LA VENTA: $2675,00
PRODUCTO CON MAYOR GASTO: Laptop ($1700,00)
-----
BUILD SUCCESS
-----
Total time: 1.382 s
Finished at: 2026-05-03T00:32:46-05:00
-----
```

El código compila de manera correcta en JAVA pero aquí volvemos a utilizar una ruta para el archivo txt ya que el mismo se encontraba en otra carpeta y no con el programa fuente.

9) Archivo de texto.

A screenshot of a text editor window titled 'productos.txt'. The editor has a dark theme and a menu bar with 'Archivo', 'Editar', and 'Ver'. The toolbar includes icons for font size (H1), list view, bold (B), italic (I), and other text formatting options. The main text area contains the following lines: 'Laptop 850 2', 'Mouse 15 5', 'Teclado 30 3', 'Monitor 180 2', and 'Ventilador 50 9'. The status bar at the bottom shows 'Ln 5, Col 16', '66 caracteres.', 'Texto sin formatear', '100%', 'Unix (LF)', and 'UTF-8'.

Visualización del archivo txt.

Conclusiones

- Se logró la integración de persistencia de datos mediante el uso de archivos `.txt`, permitiendo que el sistema procese nombres y calificaciones sin necesidad de ingresarlos manualmente en cada ejecución.
- La implementación en ambos lenguajes (C++ y Java) demostró que, aunque la sintaxis varía, la lógica fundamental de apertura, lectura y cierre de flujos de datos es consistente para el análisis de información.
- El uso de validaciones de existencia de archivos, como `archivo.exists()` en Java o `!archivo.is_open()` en C++, resultó esencial para evitar fallos críticos en el programa cuando el recurso externo no se encuentra en el directorio correcto.

Recomendaciones

- **Organización del Directorio:** Es fundamental asegurar que los archivos de texto (`estudiantes.txt`, `productos.txt`) se encuentren en la misma carpeta que el código fuente o el ejecutable para evitar errores de tipo "No encuentra archivo".
- **Optimización en Java:** Para proyectos de ingeniería en software, se recomienda el uso de extensiones como *Extension Pack for Java* en Visual Studio Code, lo

que facilita la compilación automática y la detección de errores de sintaxis en tiempo real.

- **Escalabilidad del Código:** Para el Ejercicio 2 de productos, se sugiere implementar funciones independientes que calculen el subtotal y el total, facilitando así la identificación del producto con el precio mayor de forma modular.